

Phase 2: Backend & Database Foundation

Our goal for Phase 2 is to move away from static mock data and build a robust backend architecture. This involves setting up a real database to store users and trades, implementing secure authentication, and creating the API layer to serve this data to our frontend.

User Review Required

[!IMPORTANT] **Architecture Decision:** The original Phase 1 plan mentioned a separate NestJS backend. However, since we are using Next.js 14 App Router, it is often much faster and perfectly scalable to use **Next.js Route Handlers (API Routes) + Server Actions** for a monolithic architecture. This plan assumes we will use the Next.js monolithic approach for speed and simplicity. Please confirm if this is acceptable, or if you strictly prefer a separate NestJS microservice.

[!IMPORTANT] **Database Provider:** We will use **PostgreSQL** as our database, accessed via **Prisma ORM**. You will need to provision a free database url (e.g., from Supabase or Neon.tech) and add it to a **.env** file before we begin execution.

Open Questions

[!NOTE]

1. **Authentication Providers:** I plan to integrate **NextAuth.js (Auth.js)**. Should we start with standard Email/Password authentication, or do you want to include OAuth providers like Google/GitHub immediately?
2. **Database Hosting:** Do you already have a PostgreSQL connection string ready to use, or do you need instructions on how to set one up for free using Supabase or Neon?

Proposed Changes

1. Database & ORM (Prisma)

Initialize Prisma ORM and design the core PostgreSQL database schema.

[NEW] **prisma/schema.prisma**

Define the initial data models:

- **User:** ID, Email, PasswordHash, Name, SubscriptionTier, Settings
- **Account/Portfolio:** ID, UserID, BrokerName, InitialBalance, CurrentBalance
- **Trade:** ID, AccountID, Symbol, Instrument (Stock/Option), Side (Buy/Sell), EntryPrice, ExitPrice, Quantity, EntryTime, ExitTime, PnL, Strategy, Tags
- **Insight:** ID, UserID, Type (Behavioral/Risk), Severity, Description, Date

2. Authentication (NextAuth.js)

Set up secure session management and authentication routes.

[NEW] `src/app/api/auth/[...nextauth]/route.ts`

Configure NextAuth with PrismaAdapter and credentials/OAuth providers.

[NEW] `src/middleware.ts`

Implement Next.js middleware to protect the `/dashboard`, `/journal`, `/analytics`, `/strategies`, and `/risk` routes from unauthenticated users.

[NEW] `src/app/(auth)/login/page.tsx`**[NEW]** `src/app/(auth)/register/page.tsx`

Create beautifully styled login and registration pages that match the TradeMind AI aesthetic.

3. API Layer & Data Fetching

Replace static functions in `src/lib/analytics.ts` and `src/data/mockTrades.ts` with real database queries.

[NEW] `src/lib/db.ts`

Instantiate the global Prisma client to prevent connection exhaustion in development.

[NEW] **Server Actions & API Routes**

Create functions to securely fetch trades and portfolio data for the authenticated user:

- `getTrades(userId, filters)`
- `getUserMetrics(userId)`
- `saveTrade(tradeData)`

4. Frontend Integration

Modify existing dashboard components to accept data via props or fetch it directly using Server Components instead of relying on `mockTrades.ts`.

[MODIFY] `src/app/(dashboard)/**/*.tsx`

Convert pages to Server Components that securely fetch data from the database using the user's session ID, then pass the data to the client-side chart components.

Verification Plan

Automated Tests

```
# Verify Prisma schema
npx prisma validate
```

```
# Push schema to database
npx prisma db push

# Build verification
npm run build
```

Manual Verification

1. Attempt to access `/dashboard` without being logged in and verify redirect to `/login`.
2. Register a new user account via the UI.
3. Log in successfully and see an empty, real dashboard (0 trades).
4. Manually inject a few test trades into the PostgreSQL database using Prisma Studio (`npx prisma studio`).
5. Verify the dashboard charts instantly reflect the new real data.